

BÀI TẬP THỰC HÀNH

LÀM SẠCH DỮ LIỆU VỚI PYSPARK

1. Mục tiêu thực hành

- Giải thích được vai trò của data cleaning trong pipeline phân tích dữ liệu lớn.
- Định nghĩa và sử dụng Spark Schema khi đọc dữ liệu.
- Phân biệt transformation và action; hiểu tính immutable và lazy processing.
- Đọc, làm sạch và ghi dữ liệu CSV/Parquet bằng PySpark.
- Sử dụng các thao tác DataFrame: filter, select, withColumn, drop, split, cast.
- Sử dụng when/otherwise và UDF cho các quy tắc làm sạch phức tạp.
- Hiểu partition, monotonically_increasing_id, caching, shuffle và broadcast join.
- Xây dựng một pipeline làm sạch dữ liệu và kiểm tra dữ liệu sau làm sạch.

2. Dữ liệu sử dụng

Tập dữ liệu	Nội dung	Mục đích
DallasCouncilVoters.csv.gz	Danh sách cử tri/họ tên/chức danh	String cleaning, filter, UDF, validation
DallasCouncilVotes.csv.gz	Dữ liệu phiếu biểu quyết	Pipeline làm sạch và phân tích
AA_DFW_2014...2017.csv.gz	Dữ liệu chuyến bay từ DFW	Schema, union, Parquet, mini project

3. Nội dung thực hành

Bài 1. Khởi tạo môi trường PySpark

Mục tiêu của phần này là tạo SparkSession và kiểm tra môi trường trước khi xử lý dữ liệu.

Bước 1. Cài PySpark nếu cần

```
import importlib.util
import subprocess
import sys

if importlib.util.find_spec("pyspark") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "pyspark"])
```

Bước 2. Khởi tạo SparkSession

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *

spark = (
    SparkSession.builder
    .appName("CleaningDataWithPySpark")
    .master("local[*]")
    .getOrCreate()
)

sc = spark.sparkContext
```

```
print("Spark version:", spark.version)
print("Master:", sc.master)
```

Lưu ý: chạy trong local mode, khi triển khai trên cluster, phần lớn code xử lý DataFrame vẫn giữ nguyên.

Bài 2. Data Cleaning và ví dụ cơ bản

Data Cleaning là quá trình chuẩn bị dữ liệu thô trước khi đưa vào các bước xử lý và phân tích. Các công việc thường gặp gồm chuẩn hóa chuỗi, tính toán lại giá trị, loại dữ liệu rác và xử lý dữ liệu thiếu.

2.1. Tạo dữ liệu thô minh họa

```
raw_data = [
    ("Smith, John", 37, "Dallas"),
    ("Wilson, A.", 59, "Chicago"),
    (None, 215, None)
]

raw_df = spark.createDataFrame(raw_data, ["name", "age_years", "city"])
raw_df.show(truncate=False)
```

2.2. Làm sạch dữ liệu

Ta loại dòng không có tên, tách họ/tên, chuyển tuổi từ năm sang tháng và ánh xạ thành phố sang mã bang.

```
city_to_state = (
    F.when(F.col("city") == "Dallas", "TX")
    .when(F.col("city") == "Chicago", "IL")
)

clean_demo_df = (
    raw_df
    .filter(F.col("name").isNotNull())
    .withColumn("last_name", F.trim(F.split("name", ",").getItem(0)))
    .withColumn("first_name", F.trim(F.split("name", ",").getItem(1)))
    .withColumn("age_months", F.col("age_years") * 12)
    .withColumn("state", city_to_state)
    .select("last_name", "first_name", "age_months", "state")
)

clean_demo_df.show(truncate=False)
```

Bài 3. Spark Schema

Schema xác định tên cột, kiểu dữ liệu và khả năng nhận giá trị null. Khi cấu trúc dữ liệu đã biết, khai báo schema rõ ràng thường tốt hơn inferSchema vì tránh quét dữ liệu nhiều lần và giúp kiểm soát kiểu dữ liệu.

```
people_schema = StructType([
    StructField("name", StringType(), False),
    StructField("age", IntegerType(), False),
    StructField("city", StringType(), False)
])
```

3.1. Schema cho dữ liệu chuyến bay

```
flight_schema = StructType([
    StructField("Date (MM/DD/YYYY)", StringType(), True),
```

```

    StructField("Flight Number", StringType(), True),
    StructField("Destination Airport", StringType(), True),
    StructField("Actual elapsed time (Minutes)", IntegerType(), True)
])

flight_2014 = spark.read.csv(
    "Datasets/AA_DFW_2014_Departures_Short.csv.gz",
    header=True,
    schema=flight_schema
)

flight_2014.printSchema()
flight_2014.show(5)

```

Nếu dữ liệu có kích thước lớn và cấu trúc đã biết, nên ưu tiên schema tường minh thay vì inferSchema=True.

Bài 4. Immutability và Lazy Processing

4.1. Immutability

Spark DataFrame là immutable. Transformation không sửa trực tiếp DataFrame cũ mà tạo một DataFrame logic mới.

```

df_original = spark.createDataFrame([
    ("Alice", 24),
    ("Bob", 30)
], ["name", "age"])

df_new = df_original.withColumn("age_next_year", F.col("age") + 1)

df_original.show()
df_new.show()

```

4.2. Lazy Processing

Các lệnh withColumn, select, filter, drop là transformations. Spark trì hoãn thực thi đến khi có action như show, count, collect hoặc write.

```

lazy_df = flight_2014.withColumn(
    "airport",
    F.lower(F.col("Destination Airport"))
).drop("Destination Airport")

# Action
lazy_df.show(5)

```

4.3. Quan sát kế hoạch thực thi

```

lazy_df.filter(F.col("airport") == "hnl").select("airport").explain()

```

Bài 5. Làm việc với Parquet

Parquet là định dạng dạng cột, lưu schema và hỗ trợ predicate pushdown. Đây là định dạng phù hợp cho dữ liệu trung gian trong nhiều pipeline Spark.

5.1. Gộp nhiều năm dữ liệu

```

files = [
    "AA_DFW_2014_Departures_Short.csv.gz",
    "AA_DFW_2015_Departures_Short.csv.gz",

```

```
"AA_DFW_2016_Departures_Short.csv.gz",
"AA_DFW_2017_Departures_Short.csv.gz"
]

flight_dfs = [
    spark.read.csv(f"Datasets/{f}", header=True, schema=flight_schema)
    for f in files
]

flights_all = flight_dfs[0]
for df in flight_dfs[1:]:
    flights_all = flights_all.unionByName(df)
```

5.2. Ghi và đọc lại Parquet

```
flights_all.write.mode("overwrite").parquet("AA_DFW_ALL.parquet")

flights_parquet = spark.read.parquet("AA_DFW_ALL.parquet")
flights_parquet.printSchema()
```

Bài 6. DataFrame Column Operations

Các thao tác cột là phần cốt lõi của data cleaning trong PySpark.

6.1. Đọc dữ liệu Dallas Council Voters

```
voter_df = spark.read.csv(
    "Datasets/DallasCouncilVoters.csv.gz",
    header=True,
    inferSchema=True
)

voter_df.printSchema()
voter_df.show(10, truncate=False)
```

6.2. Lọc dữ liệu

```
voter_clean = (
    voter_df
    .filter(F.col("VOTER_NAME").isNotNull())
    .filter(F.length("VOTER_NAME").between(1, 20))
    .filter(~F.col("VOTER_NAME").contains("_"))
)
```

6.3. Biến đổi chuỗi

```
voter_string = (
    voter_df
    .filter(F.col("VOTER_NAME").isNotNull())
    .withColumn("NAME_UPPER", F.upper("VOTER_NAME"))
    .withColumn("NAME_PARTS", F.split("VOTER_NAME", " "))
    .withColumn("PART_COUNT", F.size("NAME_PARTS"))
)

voter_string.select(
    "VOTER_NAME", "NAME_UPPER", "NAME_PARTS", "PART_COUNT"
).show(10, truncate=False)
```

6.4. Tách first name và last name

```
voter_names = (
    voter_df
    .filter(F.col("VOTER_NAME").isNotNull())
    .withColumn("name_parts", F.split(F.trim("VOTER_NAME"), " "))
    .withColumn("first_name", F.col("name_parts").getItem(0))
    .withColumn(
        "last_name",
        F.col("name_parts").getItem(F.size("name_parts") - 1)
    )
)

voter_names.select(
    "VOTER_NAME", "first_name", "last_name"
).show(10, truncate=False)
```

Bài 7. Conditional Column Operations

F.when() và otherwise() cho phép tạo logic điều kiện trực tiếp trên cột DataFrame.

```
age_df = spark.createDataFrame([
    ("Alice", 14),
    ("Bob", 18),
    ("Candice", 38)
], ["Name", "Age"])

classified_df = age_df.withColumn(
    "Age_Group",
    F.when(F.col("Age") >= 18, "Adult")
    .otherwise("Minor")
)

classified_df.show()
```

7.1. Nhiều nhánh điều kiện

```
multi_df = age_df.withColumn(
    "Category",
    F.when(F.col("Age") < 13, "Child")
    .when(F.col("Age") < 18, "Teen")
    .otherwise("Adult")
)

multi_df.show()
```

Bài 8. User Defined Functions - UDF

UDF dùng khi logic xử lý khó biểu diễn bằng các hàm có sẵn của Spark. Tuy nhiên, nếu có thể dùng built-in Spark functions thì nên ưu tiên built-in functions để Spark tối ưu tốt hơn.

8.1. UDF đảo chuỗi

```
def reverse_string(text):
    if text is None:
        return None
    return text[::-1]

udf_reverse = F.udf(reverse_string, StringType())

voter_udf = voter_df.withColumn(
```

```
"ReverseName",
  udf_reverse("VOTER_NAME")
)

voter_udf.select("VOTER_NAME", "ReverseName").show(10, truncate=False)
```

8.2. UDF đếm số từ trong tên

```
def name_word_count(name):
    if name is None:
        return 0
    return len(name.split())

udf_name_word_count = F.udf(name_word_count, IntegerType())

voter_udf = voter_udf.withColumn(
    "NameWordCount",
    udf_name_word_count("VOTER_NAME")
)
```

Bài 9. Partitioning và tạo ID

DataFrame được chia thành các partition để Spark có thể xử lý song song. Vì vậy, ID tuần tự kiểu 0,1,2,... không tự nhiên trong môi trường phân tán.

9.1. Xem số partition

```
print(voter_df.rdd.getNumPartitions())
```

9.2. monotonically_increasing_id()

```
voter_with_id = voter_df.withColumn(
    "ID",
    F.monotonically_increasing_id()
)

voter_with_id.select("ID", "VOTER_NAME").show(10, truncate=False)
```

Lưu ý: ID sinh bởi `monotonically_increasing_id()` là duy nhất và tăng, nhưng không đảm bảo liên tục. Không nên xem đây là khóa tuần tự như trong hệ quản trị CSDL

Bài 10. Caching

Caching hữu ích khi một DataFrame được dùng lại nhiều lần. Không nên cache mọi DataFrame vì cache cũng tiêu tốn bộ nhớ/tài nguyên.

```
cached_df = voter_df.cache()

# Materialize cache
row_count = cached_df.count()

print("Rows:", row_count)
print("Is cached:", cached_df.is_cached)

cached_df.unpersist()
```

Bài 11. Tối ưu nhập dữ liệu

Một số kỹ thuật thực hành:

- Định nghĩa schema rõ ràng khi biết cấu trúc dữ liệu.

- Đọc nhiều file có cấu trúc giống nhau bằng wildcard.
- Giữ kích thước file tương đối cân bằng.
- Chuyển dữ liệu trung gian sang Parquet khi phù hợp.

```
wildcard_path = "Datasets/AA_DFW_*_Departures_Short.csv.gz"

flights_wildcard = spark.read.csv(
    wildcard_path,
    header=True,
    schema=flight_schema
)

print("Số dòng:", flights_wildcard.count())
```

Bài 12. Execution Plan, Shuffle và Broadcast

12.1. Execution Plan

```
voter_df.select("VOTER_NAME").distinct().explain()
```

12.2. Shuffle

Shuffle là quá trình di chuyển dữ liệu giữa các partition/worker. Các thao tác groupBy, distinct, join, repartition thường có thể phát sinh shuffle.

12.3. repartition và coalesce

```
print("Ban đầu:", flights_wildcard.rdd.getNumPartitions())

repart_df = flights_wildcard.repartition(4)
print("repartition(4):", repart_df.rdd.getNumPartitions())

coalesce_df = repart_df.coalesce(2)
print("coalesce(2):", coalesce_df.rdd.getNumPartitions())
```

12.4. Broadcast Join

```
airport_info = spark.createDataFrame([
    ("HNL", "Honolulu"),
    ("OGG", "Kahului"),
    ("SFO", "San Francisco"),
    ("LAS", "Las Vegas"),
    ("DTW", "Detroit")
], ["airport_code", "airport_name"])

joined_broadcast = flights_wildcard.join(
    F.broadcast(airport_info),
    flights_wildcard["Destination Airport"] == airport_info["airport_code"],
    "left"
)
```

Bài 13. Data Pipeline

Một pipeline làm sạch dữ liệu có thể tổ chức theo chuỗi:

Input → Validation → Cleaning → Transformation → Analysis → Output

13.1. Pipeline mẫu với Dallas Council Votes

```

votes_df = spark.read.csv(
    "Datasets/DallasCouncilVotes.csv.gz",
    header=True,
    inferSchema=True
)

votes_clean = (
    votes_df
    .withColumnRenamed("VOTER NAME", "VOTER_NAME")
    .withColumnRenamed("VOTE CAST", "VOTE_CAST")
    .filter(F.col("VOTER_NAME").isNotNull())
    .withColumn("VOTER_NAME", F.trim(F.col("VOTER_NAME")))
    .withColumn("VOTER_NAME_UPPER", F.upper("VOTER_NAME"))
    .withColumn("ROW_ID", F.monotonically_increasing_id())
)

votes_clean.groupBy("VOTE_CAST").count().show()

```

13.2. Xuất dữ liệu sạch

```

votes_clean.write.mode("overwrite").parquet(
    "DallasCouncilVotes_Clean.parquet"
)

```

14. Data Validation

Validation kiểm tra dữ liệu có tuân thủ format và rule mong đợi hay không. Có thể kiểm tra số dòng, số cột, kiểu dữ liệu, miền giá trị hoặc so sánh với bảng tham chiếu.

14.1. Validation bằng join

```

valid_titles = spark.createDataFrame([
    ("Mayor",),
    ("Councilmember",)
], ["TITLE"])

verified_voters = voter_df.join(
    F.broadcast(valid_titles),
    on="TITLE",
    how="inner"
)

```

14.2. Tìm dữ liệu không hợp lệ bằng left_anti

```

invalid_voters = voter_df.join(
    F.broadcast(valid_titles),
    on="TITLE",
    how="left_anti"
)

invalid_voters.show(10, truncate=False)

```

Lưu ý: Trong pipeline thực tế, nên lưu riêng invalid rows để kiểm tra thay vì xóa âm thầm.

BÀI TẬP MỞ RỘNG

Mục tiêu: xây dựng một pipeline hoàn chỉnh trên 4 tệp chuyến bay 2014-2017.

Yêu cầu

1. Đọc 4 file bằng schema cố định.

2. Gộp dữ liệu bằng unionByName.
3. Chuẩn hóa tên cột.
4. Loại bản ghi có thời gian bay null hoặc ≤ 0 .
5. Chuẩn hóa airport code thành uppercase.
6. Chuyển ngày sang kiểu date và tạo cột year.
7. Tính số chuyến, thời gian trung bình, nhỏ nhất và lớn nhất theo năm và destination.
8. Ghi kết quả ra Parquet.

Code tham khảo

```
clean_flights = (
    flights_wildcard
    .withColumnRenamed("Date (MM/DD/YYYY)", "flight_date")
    .withColumnRenamed("Flight Number", "flight_number")
    .withColumnRenamed("Destination Airport", "destination")
    .withColumnRenamed(
        "Actual elapsed time (Minutes)",
        "elapsed_minutes"
    )
    .filter(F.col("elapsed_minutes").isNotNull())
    .filter(F.col("elapsed_minutes") > 0)
    .withColumn("destination", F.upper(F.trim("destination")))
    .withColumn(
        "flight_date_parsed",
        F.to_date("flight_date", "MM/dd/yyyy")
    )
    .withColumn("year", F.year("flight_date_parsed"))
)

flight_stats = (
    clean_flights
    .groupBy("year", "destination")
    .agg(
        F.count("*").alias("num_flights"),
        F.round(F.avg("elapsed_minutes"), 2).alias("avg_minutes"),
        F.min("elapsed_minutes").alias("min_minutes"),
        F.max("elapsed_minutes").alias("max_minutes")
    )
    .orderBy("year", F.desc("num_flights"))
)

flight_stats.show(30)
```

Câu hỏi phân tích

- Destination nào có nhiều chuyến bay nhất trong từng năm?
- Destination nào có thời gian bay trung bình lớn nhất?
- Có bản ghi nào có elapsed_minutes bất thường không?
- Nếu chuyển pipeline sang Parquet, những bước nào có thể chạy hiệu quả hơn?
- Có nên cache clean_flights trước khi chạy nhiều phép phân tích khác nhau? Giải thích.

4. Bài tập về nhà

Bài 1. Schema

- Tạo schema cho product_id, product_name, price, quantity.
- Đọc một CSV nhỏ bằng schema này.
- Kiểm tra printSchema().

Bài 2. Cleaning Voter

- Loại VOTER_NAME null.
- trim khoảng trắng.
- đổi uppercase.
- tách tên thành array.
- đếm số thành phần của tên.

Bài 3. Conditional Column

- Tạo cột flight_type: <120 phút = Short; 120-240 = Medium; >240 = Long.
- Đếm số chuyến theo flight_type.

Bài 4. Parquet

- Đọc 4 file chuyến bay.
- Union dữ liệu.
- Ghi Parquet.
- Đọc lại và so sánh schema/số dòng.

Bài 5. Cache

- Chạy cùng một phép groupBy trước khi cache.
- Cache DataFrame và chạy lại.
- Unpersist sau khi dùng.
- Nhận xét thời gian.

Bài 6. Broadcast Join

- Tạo bảng lookup airport.
- Join bình thường và broadcast join.
- Dùng explain() để so sánh execution plan.

Bài 7. Validation

- Tạo danh sách giá trị VOTE_CAST hợp lệ.
- Giữ valid rows bằng join.
- Lấy invalid rows bằng left_anti.

5. Câu hỏi ôn tập

1. Vì sao schema tường minh có thể cải thiện hiệu năng đọc dữ liệu?

2. Immutable DataFrame có ý nghĩa gì?
3. Transformation khác action như thế nào?
4. Vì sao Parquet phù hợp cho phân tích dữ liệu lớn?
5. Khi nào nên dùng UDF? Khi nào không nên dùng?
6. `monotonically_increasing_id()` có tạo ID liên tục hay không?
7. Cache có phải lúc nào cũng làm chương trình nhanh hơn không?
8. Shuffle là gì và tại sao cần hạn chế?
9. Broadcast join thích hợp trong trường hợp nào?
10. `left_anti` join hữu ích như thế nào trong data validation?

Tổng kết

Sau bài thực hành, sinh viên cần nắm được quy trình làm sạch dữ liệu lớn bằng PySpark, từ khai báo schema và đọc dữ liệu đến cleaning, validation, tối ưu thực thi và xuất dữ liệu sạch.

Chuỗi xử lý nên ghi nhớ: Read → Validate → Clean → Transform → Analyze → Write

---o0o---

Hết